

Object Relational Model (ORM)

Learning Goals

- Explain ORM as a translator between PHP objects and database rows.
- Compare PDO and Eloquent for finding, adding, changing, and removing a student.
- Describe what Eloquent does for you: understand the PHP call, build the SQL, and return objects.

Important, but Only the Basics Today

ORM is an important topic in real-world Laravel projects, but it can become advanced quickly.

- In this lesson, we learn only the **basic idea**: objects and database rows can match.
- You have already used ORM: the `Student` model from the Laravel lesson is an Eloquent ORM model.
- A real project needs more knowledge, such as relationships, query performance, and safe data changes.
- Those optional, advanced topics come later in the optional lessons **3-1 and 3-2**.

For now, think of ORM as a helpful translator. That is enough to get started.

Two Different Worlds

Our program and our database speak different languages.

In PHP	In MySQL
a <code>Student</code> object	a row in the <code>students</code> table
<code>\$student->name</code>	the <code>name</code> column
an array of objects	many rows in a table

PHP likes **objects**. MySQL likes **tables, rows, and columns**.

ORM Is a Translator

ORM means **Object-Relational Mapping**.

- **Object**: a PHP thing such as one Student
- **Relational**: a database made of related tables
- **Mapping**: matching one thing to another

Think of ORM as a friendly translator between PHP and MySQL.

Student object <----- ORM -----> students table

A Simple Matching Game

Imagine a table is a class roster.

<code>students</code> table	<code>Student</code> object
one row	one student object
<code>name</code> column	<code>\$student->name</code>
<code>age</code> column	<code>\$student->age</code>
<code>id</code> column	<code>\$student->id</code>

One row such as `Ada, 20, CS` becomes a `Student` object that PHP can use.

Without an ORM

PDO asks MySQL directly with SQL:

```
$statement = $pdo->prepare(  
    'SELECT * FROM students WHERE id = ?'  
);  
$statement->execute( [$id] );  
$row = $statement->fetch();  
  
echo $row['name'];
```

This is completely valid. You write the SQL and receive an array.

With an ORM

With Laravel's ORM, we describe the student object we want:

```
$student = Student::find($id);  
  
echo $student->name;
```

Eloquent creates and runs the SQL behind the scenes, then gives us a `Student` object.

Same Job, Different Conversation

Task	PDO	Eloquent ORM
Find student 5	SQL + <code>fetch()</code>	<code>Student::find(5)</code>
Get the name	<code>\$row['name']</code>	<code>\$student->name</code>
Add a student	<code>INSERT INTO</code> <code>...</code>	<code>Student::create(...)</code>
Change a student	<code>UPDATE ...</code>	<code>\$student->update(...)</code>
Remove a student	<code>DELETE FROM</code> <code>...</code>	<code>\$student->delete()</code>

ORM does not remove the database. It gives us an easier way to

What Does the ORM Do for Us?

When we write:

```
Student::where('major', 'CS')->get();
```

the ORM does three jobs:

1. Understands our PHP instruction.
2. Builds and runs the matching SQL query.
3. Turns the returned rows into `Student` objects.

We still need to know what data we want. The ORM handles much of the translation.

Laravel's ORM Has a Name: Eloquent

In Laravel, the ORM is called **Eloquent**.

```
class Student extends Model
{
    // This model represents the students table.
}
```

- `Student` is the PHP model class.
- Eloquent connects it to the `students` table.
- Each `Student` object represents one student row.

Next, we will use Eloquent to create, find, update, and delete student records.